Programação Orientada a Objetos 2026/2027

Exercícios

Ficha Nº 4

Herança e polimorfismo
Primeira abordagem – cenários em que as classes não são proprietárias dos objetos que usam, e sem uso de coleções.
Sintaxe de herança e comportamento polimórfico.

1. Imóveis / Imobiliária – Exercício preparatório, feito/apresentado de forma interativa com o professor

Uma agência imobiliária tem, para vender, apartamentos e lojas comerciais. Os apartamentos são caracterizados pelo preço, área, andar e número de assoalhadas (tipologia: número de quartos + sala). As lojas são caracterizadas pelo preço e área, situando-se sempre no rés-do-chão. Qualquer destes imóveis é caracterizado também por um código, gerado automaticamente, sendo formado pela palavra "apartamento" ou "loja", seguida por um inteiro que corresponde à ordem pela qual o objeto foi gerado (a sequência da parte numérica é a mesma – partilhada – para todos os tipos de imóveis). Os apartamentos têm um preço inicial que é sempre 10 x área. As lojas têm um preço inicial que é 15 x área. Tanto os apartamentos como as lojas devem permitir:

- Obter o código
- Obter o preço
- Obter uma *string* com a descrição de todos os dados
- Fazer aumentar os preços (infelizmente). Os apartamentos aumentam N% em que N é o número de quartos+sala. As lojas aumentam sempre 1%.

> ***O texto seguinte não faz normalmente parte de um enunciado (e muito menos de um exame)***.
> *Na sequência do exercício anterior deve ter ficado em condições de identificar os aspetos seguintes.*
>
> Existem aqui dois conceitos muito relacionados entre si, mas ao mesmo tempo claramente distintos: apartamentos e lojas. Aquilo que partilham é relativamente óbvio. As diferenças que existem são tanto a nível de dados como a nível de comportamento/restrições:
>
> - **Dados**: um apartamento tem assoalhadas; uma loja não.
> - **Comportamento**: uma loja está sempre no rés-do-chão; um apartamento pode estar em qualquer andar.
>
> A forma correta de modelizar esta situação em C++ será a de fazer corresponder uma classe distinta a cada entidade Apartamento e Loja, e usar herança para fazer ambas as classes herdarem aquilo que partilham a partir de uma classe base comum: ambas as classes derivam de uma classe que encapsula os dados e comportamentos partilhados.

a) Defina as classes que representam estes dois bens imobiliários usando adequadamente os mecanismos de C++ vocacionados para a reutilização de código e extensão de conceitos da forma ***conceito base – conceito especializado***. Atenção: no enunciado existem dois tipos de imóveis, mas facilmente poderiam ser 200, e pretende-se que o programa seja expansível para novos tipos de imóvel. Ajuda: vão ser necessárias três classes.

As classes devem garantir que os seus objetos só são construídos se lhe forem passados todos os dados relevantes. Os dados devem ser validados (por exemplo, área > 0), mas sem entrar em exageros dado que o foco do exercício é outro.

b) Defina a classe Imobiliaria que representa todo o conjunto de bens imobiliários que a agência tem para vender neste edifício. Deve ser possível:

- Acrescentar imóveis à imobiliária (não são aceites códigos repetidos).
- Obter listagens de bens imobiliários: todos, por andar.
- Obter uma visão "só de leitura" de um imóvel dado o código que permita ao resto do programa obter dados (só de leitura) do imóvel.
- Obter a descrição do imóvel dado o código.
- Remover por código (remover não destrói o imóvel – apenas deixa de ser considerado na imobiliária).
- Aumentar o preço.

Deve também ter em atenção que não deve utilizar uma estrutura de dados diferente para cada conjunto de tipo de bem imobiliário (caso contrário o professor vai propor a existência de 200 tipos de imóveis) - podem mesmo existir muitos tipos diferentes – **deve unificar o armazenamento de todos os bens imobiliários numa única estrutura de dados**. Dado que ainda não foram abordadas coleções, não se pretende que use coisas como vector<> mesmo que já as conheça. Pode assumir o limite máximo de 100 imóveis nesta imobiliária.

Neste exercício os imóveis não representam artefactos internos à classe imobiliária, mas sim coisas com existência própria:

- Não é a imobiliária que constrói os edifícios – já existem.
- Se ocorrer um curto-circuito no sistema de informação da imobiliária, os edifícios continuam a existir.

Quer isto dizer que **um objeto da classe Imobiliária não controla a existência dos objetos Imóveis que conhece/usa**. Pode assumir que esses objetos pertencem à função *main()* (abordagem simplista, mas que é a única possível nesta fase da matéria).

c) Teste a funcionalidade da alínea b) criando alguns imóveis, registando-os na imobiliária, e depois procurando-os/listando-os/etc.

d) TPC (fora da aula): Acrescente o tipo de imóvel Escritório. É parecido com o apartamento mas tem sempre tipologia 2, o código tem a palavra "escritório", e o preço base é 10x a área e aumenta sempre 2%.

Na aula avance para a alínea seguinte.

e) Debata com o professor o seguinte:

1. Neste cenário, o que faz mais sentido: a imobiliária ser "dona" dos imóveis que lista? Ou será que isso não faria sentido? Porquê? Por imóvel entende-se não uma qualquer "ficha de registo" mas sim o objeto que realmente representa, no programa, o imóvel
2. Se, porventura a imobiliária fosse dona dos imóveis, o que seria diferente quanto a:
    a. Inserção dos objetos: quem e como os constrói?
    b. E se a imobiliária fosse destruída / duplicada (cópia/atribuição)?

Nesta alínea pretende-se que participe na aula e que compreenda o código apresentado pelo professor relativamente a estas hipóteses de alterações. Estas considerações serão muito importantes na matéria, mas apenas mais adiante, pretendendo-se agora consciencializar para este tipo de questão . Não se pretende que gaste mais que 10 minutos nesta alínea (não é para fazer código novo e nem sequer tem a matéria necessária – apenas para pensar e participar)

No final do exercício deverá ter

- Compreendido os casos de uso onde se aplica herança
- Compreendido o conceito de classe base e como se identifica, e o conceito de classes derivadas e como se relacionam com a classe base
- Compreendida a sintaxe envolvida na herança incluindo, sem exclusão de outros aspetos: a definição de construtores em classes derivadas, e a forma correta de aceder a dados e funcionalidades da classe base
- Entendido e aplicado o conceito de polimorfismo, e percebido o conceito de função virtual.
- Visto um exemplo de uso de dados static.

2. Clínica veterinária

Uma clínica veterinária acompanha animais de estimação de diferentes espécies. Para já, considera-se que a clínica segue **duas espécies**: Cão e Gato. Todos os animais são caracterizados por nome, peso (em kg), e data de nascimento (uma *string*). De acordo com a legislação em vigor, todos os animais de estimação são registados quando nascem, sendo-lhes atribuído um código que resulta da concatenação de um número inteiro, sequencial crescente, que começa em 1000 com um "-" e com o nome da espécie a que pertencem (exemplo: "1000-Gato"). Cada novo animal terá a parte numérica do código superior em uma unidade ao anterior, sendo esta sequência comum a todas as espécies.

Todos os dados devem poder ser obtidos a partir do exterior das classes envolvidas. Nenhum dado deve ter mais visibilidade do que a estritamente necessária. Devem ser feitas validações básicas aos dados (por exemplo: peso negativo? Isso não existe).

A clínica regista todos os animais que acompanha, mas estes não vivem na Clínica (de noite estão em casa, com os seus donos). Durante o dia estão na clínica e esta efetua algumas operações que variam consoante a espécie: comer e exercitar. Estas atividades são representadas simbolicamente pela emissão de uma mensagem (retornar uma *string*), que é depois posta no ecrã pela clínica, havendo também, eventualmente, algum outro efeito lateral, como se descreve a seguir:

Quando come, o cão produz a mensagem “biscoitos bons. obrigado” e engorda 5% do peso, até ao máximo de 20 kg; já o gato “diz” algo como “o quê, este patê reles novamente?”. Quando se exercita, o cão emite uma *string* com a expressão: “corri X quilómetros”, em que X é um valor aleatório entre 1 e 5. O gato emite a mensagem “derrubei N jarras e arranhei M pessoas”, em que N e M são valores aleatórios (0-3 e 0-4, respetivamente), e emagrece 10% do peso até ao mínimo de 2.5 kg.

A clínica deve suportar a seguinte funcionalidade:

- Registar um novo animal, o qual já existe (não nascem animais na clínica).
- Remover um animal dado o seu código.
- Obter uma visão só de leitura de um animal dado o seu código, e desde que este esteja registado na clínica. O programa poderá depois consultar (mas não alterar) os dados do animal em questão.
- Obter a descrição de um animal dado o código (a descrição deve permitir saber que tipo de animal é).
- Obter uma lista completa com a descrição de todos os animais.
- Desencadear a alimentação de todos os animais.
- Desencadear o exercício de todos os animais.

A clínica suporta apenas até 200 animais. Não deve usar coleções da biblioteca standard mesmo que já conheça o que isso é.

Escreva as classes envolvidas, cumprindo a funcionalidade pretendida. Inclua uma função *main()* com alguns animais e uma clínica para efeitos de teste. Não se dê ao trabalho de construir menus (a não ser que queira fazer isso em casa como TPC).

No final do exercício deverá ter

- Cimentado os conceitos acerca de herança e polimorfismo: onde e quando aplicar.
- Compreendido o conceito de classe base e como se identifica, e o conceito de classes derivadas e como se relacionam com a classe base
- Compreendida a sintaxe envolvida na herança incluindo, sem exclusão de outros aspetos: a definição de construtores em classes derivadas, e a forma correta de aceder a dados e funcionalidades da classe base
- Entendido e aplicado o conceito de polimorfismo, e percebido o conceito de função virtual.
- Visto um exemplo de uso de dados static.